Problemset de Práctica 01- Repaso: Montículos a Optimalidad Dinámica III

Prof. Víctor Racsó Galván Oyola

Esta lista no será evaluada, su único objetivo es permitir practicar los conceptos teóricos vistos en el curso hasta ahora: Montículos (binario, binomial y Fibonacci), Persistencia (niveles, nodos gordos, Path Copying), Retroactividad, Range Trees con Fractional Cascading (2D y 3D), y Optimalidad Dinámica (modelo BST y sus propiedades, Splay Trees, visión geométrica). Las preguntas están agrupadas en tres niveles de dificultad creciente; dentro de cada nivel, hay una pregunta por cada gran tema del curso. El nivel **Fácil** corresponde a un calentamiento directo; el nivel **Intermedio** está calibrado al nivel esperado en la parte escrita del examen parcial (comparable en alcance a las preguntas del simulacro); el nivel **Difícil** va deliberadamente más allá de lo exigible en el examen, pensado para profundizar.

Preguntas Fáciles

1. *(Montículos.)* En un max-heap con $n \geq 2$ valores **distintos**, demuestra que el segundo valor más grande (el máximo entre todos los valores salvo la raíz) siempre se encuentra en uno de los dos **hijos de la raíz** — nunca más profundo. *Ayuda: usa únicamente la propiedad de montículo (padre $\geq$ hijo) y llega a una contradicción suponiendo que no es así; no necesitas ningún ejemplo ni trazar ningún algoritmo.*

2. *(Persistencia.)* Explica, en tus propias palabras, la diferencia entre los cuatro niveles de persistencia vistos en clase (parcial, total, confluente y funcional). Además, ordénalos de menos a más permisivo y justifica por qué cada nivel efectivamente **incluye como caso particular** al anterior (es decir, por qué cualquier estructura con persistencia total ya cumple, de paso, con persistencia parcial, etc.).

3. *(Retroactividad.)* Explica la diferencia entre retroactividad **parcial** y **completa** sobre un TDA. Da un ejemplo concreto de una consulta que solo tiene sentido bajo retroactividad completa. Además, explica por qué cualquier estructura que soporte retroactividad completa automáticamente soporta retroactividad parcial también (pero no al revés).

4. *(Range Trees / Fractional Cascading.)* Buscar un mismo valor en k listas de tamaño a lo más n cuesta $O(k \lg n)$ de forma independiente, u $O(k + \lg n)$ con Fractional Cascading. Sin usar ningún valor numérico, explica **de dónde sale exactamente** cada uno de los dos términos de $O(k + \lg n)$: ¿qué operación específica del algoritmo contribuye el término $\lg n$, y qué operación contribuye el

término k? ¿Por qué este segundo término es **aditivo** ($+k$) y no **multiplicativo** ($\times k$) como en el caso independiente?

5. *(Optimalidad Dinámica.)* En la representación geométrica de una ejecución de BST vista en clase, el costo del algoritmo se mide como el **número total de puntos** generados (un punto por cada nodo tocado en cada acceso). Explica por qué esta forma de medir el costo está **dentro de un factor constante** del costo real de ejecutar el algoritmo: ¿por qué nunca hace falta tocar un mismo nodo más de $O(1)$ veces durante una única búsqueda, y por qué eso es justo lo que se necesita para que "contar puntos" sea una medida de costo válida?

Preguntas Intermedias

6. *(Montículos.)* Demuestra que INSERT en un Montículo Binomial es $O(1)$ **amortizado**, a pesar de que UNION (que INSERT usa internamente) cuesta $O(\lg n)$ en el peor caso. *Ayuda: usa el método del potencial con Φ = número de árboles binomiales presentes en el montículo (recuerda la analogía de **suma binaria** vista en clase: Φ es exactamente la cantidad de bits en 1 de n); si* INSERT *dispara una cascada de t pasos de* BINOMIAL-LINK *consecutivos, ¿cómo cambia Φ, y cuál es el costo amortizado resultante?*

7. *(Persistencia.)* Compara las técnicas de **nodos gordos** (*fat nodes*) y **Path Copying**: (a) ¿cuánto espacio adicional usa cada una, en el peor caso, por cada operación de modificación? (b) ¿Qué nivel de persistencia logra cada técnica? (c) ¿Por qué Path Copying, y no nodos gordos, es la técnica natural para lograr persistencia **confluente**?

8. *(Retroactividad.)* En un TDA donde todas las operaciones son **conmutativas** e **invertibles** (como el visto en clase), explica cómo se implementa una operación retroactiva — insertar una operación en un tiempo pasado t — sin reconstruir todo el historial posterior, y cuál es su costo. Luego da un ejemplo de un TDA real cuyas operaciones **no** sean conmutativas, y explica concretamente por qué el mismo truco fallaría ahí.

9. *(Range Trees / Fractional Cascading.)* En un Layered Range Tree 2D sobre n puntos, cada nodo del árbol de segmentos (sobre x) guarda una lista ordenada por y de los puntos en su subárbol, con **puentes** de Fractional Cascading hacia las listas de sus dos hijos.

- ¿Cuál es el costo de una consulta rectangular $[x_1, x_2] \times [y_1, y_2]$ con esta estructura, y en qué difiere del costo de un Range Tree 2D **sin** Layered/Fractional Cascading?
- Si en vez de promover **la mitad** de los elementos de cada lista hija hacia arriba promoviéramos **todos**, ¿cuánto espacio total usaría la estructura? Justifica con la suma correspondiente.

10. *(Optimalidad Dinámica.)* Considera el conjunto de puntos $P = \{(1, 1), (4, 2)\}$ (dos accesos: a la clave 1 en el tiempo 1, y a la clave 4 en el tiempo 2).

- ¿Es P arboralmente satisfecho? Justifica usando la definición vista en clase.

- Si no lo es, da **dos** puntos distintos que, agregados individualmente a P, lo vuelven arboralmente satisfecho, y explica en general qué región del plano (en términos de x y t) contiene a **todos** los puntos válidos para este propósito.
- Toma el punto $(2, 2)$ como ejemplo (agregarlo dice: en el tiempo 2, además de tocar la clave 4, el BST también tocó la clave 2). Explica por qué esto es consistente con que un BST real, al buscar la clave 4 desde la raíz, pase por el nodo 2 en el camino: ¿en qué posición relativa (ancestro, raíz, etc.) tendría que estar el nodo 2 respecto al nodo 4 en el árbol vigente en el tiempo 2?

Preguntas Difíciles

11. *(Montículos.)* Demuestra que, en un montículo de Fibonacci, cualquier nodo x de grado k tiene al menos F_{k+2} descendientes en su subárbol (contándose a sí mismo), donde F_i es el i-ésimo número de Fibonacci ($F_0 = 0, F_1 = 1$). Usa la **regla de las marcas** vista en clase para justificar por qué el i-ésimo hijo (en orden de llegada) de x tenía, en el momento en que se volvió hijo de x, grado al menos $i - 2$. Concluye que el grado máximo de cualquier nodo en un montículo de Fibonacci con n nodos es $O(\lg n)$.

12. *(Persistencia.)* En clase se usó un Segment Tree persistente, con **una versión por cada prefijo** del arreglo, para responder consultas de "k-ésimo menor en el rango $[l, r]$" restando las versiones r y $l - 1$. Usando **la misma idea** (una versión persistente por prefijo, y restar versiones), diseña una estructura para responder, de forma **offline**, consultas de "¿cuántos valores **distintos** hay en el subarreglo $A[l..r]$?".

*Ayuda: en la versión i, indexa el Segment Tree por **posición** (no por valor como en clase); al procesar la posición i, si $A[i]$ ya había aparecido antes en una posición $j < i$, primero pon en 0 la marca que dejaste en la posición j de la versión anterior, y luego agrega la marca en la posición i para crear la nueva versión.*

Justifica por qué (suma de la versión r) $-$ (suma de la versión $l-1$) da exactamente la cantidad de valores distintos en $[l, r]$.

13. *(Retroactividad.)* Considera la siguiente línea de tiempo sobre una *priority queue* (mín-heap), donde "dm" abrevia DELETE-MIN:

tiempo	1	2	3	4	5
operación	insertar(5)	dm	insertar(3)	insertar(7)	dm

- Reconstruye el estado de la cola después de cada operación y determina Q_{ahora}.
- Usando la definición de **puente** vista en clase (t' es puente si $Q_{t'} \subseteq Q_{\text{ahora}}$), determina cuál es el puente más cercano **antes** de $t = 3{,}5$.
- Calcula, usando la fórmula vista en clase, el efecto de ejecutar INSERTAR-RETROACTIVO$(3{,}5, 4)$ (insertar la clave 4 en el tiempo 3,5, entre las operaciones de los tiempos 3 y 4). ¿Cuál es el nuevo Q_{ahora}?
- Verifica tu respuesta del inciso anterior simulando directamente la línea de tiempo completa con el 4 insertado desde el principio en $t = 3{,}5$.

14. *(Range Trees / Fractional Cascading.)* En la construcción de Range Trees 3D vista en clase, para acotar $y \in [a_2, b_2]$ se usa un **nodo de separación** v_{split} (donde los caminos de búsqueda de a_2 y b_2 en el árbol de rango sobre y se separan), guardando en derecha(v_{split}) una estructura D_2 normal (acotada por $y \leq b_2$) y en izquierda(v_{split}) una estructura D_2' invertida (acotada por $y \geq a_2$).

- Explica por qué **todo** punto en el subárbol de derecha(v_{split}) ya satisface $y \geq a_2$ automáticamente.
- Usando el inciso anterior, demuestra que consultar D_2 sobre derecha(v_{split}) y D_2' sobre izquierda(v_{split}), y unir ambos resultados, da **exactamente** los puntos con $y \in [a_2, b_2]$ en todo el subárbol de v_{split} — ni de más ni de menos.
- ¿Por qué esta técnica evita el costo $O(\lg^2 n)$ que resultaría de aplicar descomposición canónica también sobre y? Explica en términos de **cuántos** nodos participan en la consulta de y bajo cada enfoque.

15. *(Optimalidad Dinámica.)* Considera la secuencia de accesos $2, 1, 3$ (en ese orden) sobre las claves $\{1, 2, 3\}$. Simula el algoritmo GREEDY **fila por fila** (una fila por cada tiempo) para construir un conjunto arboralmente satisfecho: en cada fila, agrega el punto del acceso real y, usando la definición de satisfacción arboral, determina si hace falta agregar puntos **adicionales en esa misma fila** para que el conjunto acumulado hasta ahí se mantenga arboralmente satisfecho (recuerda que no se puede modificar ninguna fila ya procesada). Muestra tu trabajo fila por fila: qué pares de puntos revisas, cuáles violan la condición, qué punto agregas en cada caso y por qué resuelve la violación. ¿Cuántos puntos usa GREEDY en total, y cuántos de ellos son "extra" (no corresponden a un acceso real de la secuencia original)?